Ders 22 Çalışma Özeti

Ders 22 Çalışma Özeti

Genel Konular

- Dersin tarihi 7 Aralık 2021 Salı. Bu kayıt bir uygulama (laboratuvar) oturumudur, çevrim içi (Zoom) yapıldı.
  - Hoca oturumu açıp asistana devrediyor. Uygulamayı asistan anlatıyor; Zoom etiketi "betul turkkol".
  - Önceki haftanın oturumunda bağlantı sorunları yaşanmış. Bu oturum kısmen onun telafisi.
  - Asıl anlatım yaklaşık 00:04'te başlar.
- Süreç oluşturma ve sonlandırma (C, Linux)
  - `fork` ve `exec` farkı, `exec` örneği.
  - `exit(0)`, `atexit` ile çıkışta çalışan fonksiyon, `exit` ile `return` farkı.
- Zombi ve öksüz (orphan) süreçler
  - `ps` ve `grep` ile Z durumundaki süreci bulma.
  - `wait` ile ebeveynin çocukları beklemesi, çıkış durumunu okuma.
- Süreçler arası iletişim (IPC)
  - İsimsiz boru (`pipe`): ebeveyn ile çocuk arasında tek yönlü iletişim. İki örnek: sayı gönderme ve istemci-sunucu dosya okuma.
  - İsimli boru (FIFO, `mkfifo`): ilişkisiz iki süreç arasında iletişim.
  - Sinyaller (signals): sinyal işleme seçenekleri, sinyal maskesi, `sigprocmask`, `sigaction`.
- Sonraki haftanın konusu: semaforlar (başka bir asistan anlatacak).

Hocanın Özellikle Vurguladığı Kısımlar

- Bu oturumu asistan anlattı. Aşağıdakiler asistanın vurgularıdır.
- `exit` ile `return` farkı doğru-yanlış sorularında çıkabilir.
- Birinci vizedeki sorulara bakan asistanın tavsiyesi: Uygulamada ara ara geçen ayrıntılardan güzel doğru-yanlış soruları çıkar. Bunları ciddiye alın.
- Sinyalleri engellemenin (block) amacı kritik kod parçasının kesilmemesidir.
  - Asistanın örneği: Hassas bir kontrol yazılımında kritik bir işlem sürerken gelen kesme istenmez.
- Sinyal ayarlarını değiştirmeden önce eski maskeyi ya da eski `sigaction` yapısını saklayın.
  - İşiniz bitince eski hâle ("fabrika ayarlarına") geri dönebilirsiniz.

Kısa Tekrar Notları

- `fork()`: Çağıranın kopyası olan bir çocuk süreç oluşturur. Ebeveyne çocuğun PID'sini, çocuğa 0 döndürür. Hata olursa -1 döner.
- `exec` ailesi: Süreç görüntüsünü yeni bir programla değiştirir. Yeni süreç oluşturmaz.
- `atexit(f)`: Süreç normal sonlanırken `f` fonksiyonunu çalıştırmak üzere kaydeder.
- `wait(&status)`: Ebeveyni bir çocuk sonlanana kadar bekletir. Sonlanan çocuğun PID'sini döndürür.
  - `WIFEXITED(status)` çocuğun normal çıkıp çıkmadığını, `WEXITSTATUS(status)` çıkış kodunu verir.
- Zombi süreç: Sonlanmış ama ebeveyni `wait` ile durumunu almamış çocuk. Süreç tablosunda kalır, `ps` çıktısında durumu `Z`.

- Öksüz süreç: Ebeveyni sonlanmış ama kendisi hâlâ çalışan çocuk.
- `pipe(fd)`: `fd[0]` okuma ucu, `fd[1]` yazma ucudur. Tek yönlüdür (half duplex). Ortak atası olan süreçler arasında kullanılır.
- `read` 0 döndürürse dosya sonu (EOF), -1 döndürürse hata vardır.
- FIFO (isimli boru): `mkfifo(yol, mod)` ile dosya sisteminde oluşturulur. İlişkisiz süreçler kullanabilir. `unlink` ile silinir.
- Sinyale üç tepki verilebilir: yok sayma (`SIG_IGN`), varsayılan eylem (`SIG_DFL`), kullanıcı tanımlı işleyici (handler).
- `sigprocmask(SIG_BLOCK, &yeni, &eski)` sinyali engeller; `SIG_SETMASK` ile eski maske geri yüklenir.
- `sigaction(SIGINT, &act, &oldact)`: `SIGINT` için yeni işleyici kurar, eskisini saklar.

Detaylı Açıklamalar

exec örneği

Ebeveyn `fork` yapar. Çocukta `PID == 0` olduğu için çocuk `exec` ile `ls` programını çalıştırır. Ebeveyn kendi mesajını basar. Çıktıların ekranda hangi sırayla görüneceği zamanlamaya bağlıdır.

`wait` ile çocukları bekleme

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int n = 3, status;
    for (int i = 0; i < n; i++)
        if (fork() == 0)
            exit(100 + i);            /* çocuk i, 100+i ile çıkar */
    pid_t wpid;
    while ((wpid = wait(&status)) > 0)
        if (WIFEXITED(status))
            printf("çocuk %d, çıkış durumu %d\n", wpid, WEXITSTATUS(status));
    return 0;
}
```

Derste gösterilen örnek bu yapıdadır: her çocuk farklı bir çıkış kodu döndürür, ebeveyn `wait` ile bunları toplar ve makrolarla basar.

`pipe` ile ebeveyn-çocuk iletişimi

```c
int fd[2];
pipe(fd);
if (fork() == 0) {          /* çocuk: yazar */
    close(fd[0]);
    int x;
    scanf("%d", &x);
    write(fd[1], &x, sizeof x);
    close(fd[1]);
} else {                    /* ebeveyn: okur */
    close(fd[1]);
    int y;
    read(fd[0], &y, sizeof y);
    close(fd[0]);
    printf("%d\n", y);
}
```

Her süreç kullanmadığı ucu kapatır. İkinci örnekte istemci bir dosya adını boruya yazar. Sunucu dosyayı açar ve içeriğini standart çıkışa (`STDOUT_FILENO`) yazar.

FIFO ile istemci-sunucu

- Sunucu iki FIFO oluşturur (`/tmp/fifo.1`, `/tmp/fifo.2`): biri okuma, biri yazma için.
- İstemci aynı yollardaki FIFO'ları açar. Dosya adını bir FIFO'ya yazar, dosya içeriğini diğerinden okur.
- FIFO'yu açan süreç, karşı uç açılana kadar bekler.
- İş bitince FIFO dosyaları `unlink` ile silinir.

Sinyal maskesi ve `sigaction`

```
sigset_t newmask, oldmask;
sigemptyset(&newmask);
sigaddset(&newmask, SIGINT);
sigprocmask(SIG_BLOCK, &newmask, &oldmask);   /* SIGINT'i engelle */
/* kritik kod */
sigprocmask(SIG_SETMASK, &oldmask, NULL);     /* eski maskeye dön */

void ouch(int sig) { printf("SIGINT alındı\n"); }

struct sigaction act;
act.sa_handler = ouch;          /* SIG_IGN yazılırsa sinyal yok sayılır */
sigemptyset(&act.sa_mask);
act.sa_flags = 0;
sigaction(SIGINT, &act, NULL);
```

Derste Ctrl+C'ye basıldığında program durmadı, yalnız işleyicinin mesajı basıldı. `sa_handler` yerine `SIG_IGN` verilince Ctrl+C hiçbir etki yapmadı. Ctrl+Z'ye bir işleyici atanmadığı için program Ctrl+Z ile durduruldu.

Ders sonrası notlar

- Asistan öksüz süreçleri zombinin bir türü olarak anlattı. İkisi farklıdır.
  - Zombi: çocuk sonlanmıştır ama ebeveyn `wait` yapmamıştır.
  - Öksüz: ebeveyn sonlanmıştır, çocuk hâlâ çalışır. Öksüz süreci `init` (ya da `systemd` veya bir alt toplayıcı süreç) sahiplenir ve sonlanınca durumunu toplar. Bu yüzden öksüz süreç zombi olarak kalmaz.
  - Derste "zombi 2" örneğinde zombi oluşmaması bu yüzden beklenen sonuçtur. Ebeveyn çıkınca kabuk komut istemini geri verir; çocuk arka planda öksüz olarak çalışmaya devam eder (`ps -o pid,ppid,stat` ile görülebilir). Denemede çocuk `R` durumunda göründü, `PPID` sütununda `systemd --user` vardı.
- Zombi süreç zaten sonlanmıştır, `kill` ile "öldürülemez". Kaldırmak için ebeveynin `wait` yapması gerekir. Ebeveyn öldürülürse zombi `init`'e geçer ve `init` onu toplar. Denemede çocuk `ps` çıktısında `Z` (`<defunct>`) göründü; yalnız ebeveyne `kill` gönderilince tablodan kalktı.
- Asistan `exec` için "çocuğun kendi çocuğunu oluşturduğu" dedi. `exec` yeni süreç oluşturmaz; çağıran sürecin görüntüsünü değiştirir ve PID aynı kalır.
- Asistan "çıkış durumu ebeveyne ulaşana kadar ebeveyn çıkamaz, kural bu" dedi. Böyle bir kural yoktur. Ebeveyn istediği an sonlanabilir; o zaman çocuklar öksüz kalır. `wait` yalnızca ebeveyni bir çocuk sonlanana kadar bekletir.
- Asistan `wait(NULL)` için "bütün çocukları bekler" dedi. `wait(NULL)` herhangi bir çocuğun sonlanmasını bekler ve durumu yok sayar. Bütün çocukları beklemek için `wait`, -1 döndürene kadar döngüde çağrılır.
- Asistan "`exit` sistem çağrısı, `return` bir komut" dedi. Daha doğrusu: `exit()` bir C kütüphane fonksiyonudur. `atexit` ile kayıtlı fonksiyonları çalıştırır, `stdio` tamponlarını boşaltır, sonra `_exit` sistem çağrısını yapar. `main`'den `return` etmek `exit`'i çağırmakla aynı etkiyi yapar. Başka bir fonksiyondan `return` ise yalnız o fonksiyondan döner.
- `fork` ile oluşan çocuk, ebeveynin `atexit` kayıtlarını devralır. Çocuk `exit` ile çıkarsa aynı fonksiyon çocukta da çalışır. Bunu istemiyorsanız çocukta `_exit` kullanın. Denemede `fork`'tan önce kaydedilen fonksiyon iki farklı PID ile iki kez çalıştı.
- Ctrl+C `SIGINT` (kesme) sinyali gönderir; asistan bunu bir an "termination sinyali" diye andı. Sonlandırma sinyali `SIGTERM`'dür. Ctrl+Z `SIGTSTP` gönderir.

- Engellenen (blocked) sinyal yok sayılmaz; beklemede (pending) kalır ve engel kalkınca teslim edilir. Sinyali yok saymak için `SIG_IGN` kullanılır. Denemede `SIGINT` engelliyken `raise(SIGINT)` çağrıldı; `sigpending` sinyalin beklemede olduğunu gösterdi, eski maskeye dönülünce sinyal teslim edildi ve süreç sonlandı.
- FIFO'yu sunucu oluşturuyorsa istemci önce çalıştırılmamalıdır. FIFO dosyası yokken `open` hata verir. Derste bir öğrenci bunu fark etti.
- FIFO dosyası `mkfifo`'ya verilen yolda oluşur. Derste `/tmp` altında oluşması örnek kodda o yolun seçilmesinden kaynaklanır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.